

Programming Project report

Galaga

Raúl Portugués & Diego García

Group 88

Computer Science

Abstract

This project tries to emulate the game Galaga. After following the steps indicated on the project requisites, a final functional version has been reached. The code has been designed taking into account future maintainability, code understandability and new game functions.

Table of contents

1. Classes Design
2. Most Relevant Fields and Methods
3. Most Relevant Algorithms
4. Performed work
5. Conclusions
6. Personal Comments

1. Classes design

We have created 14 **classes** to achieve a better project organisation. We used inheritance as well as abstract classes, information hiding, encapsulation and other object oriented techniques.

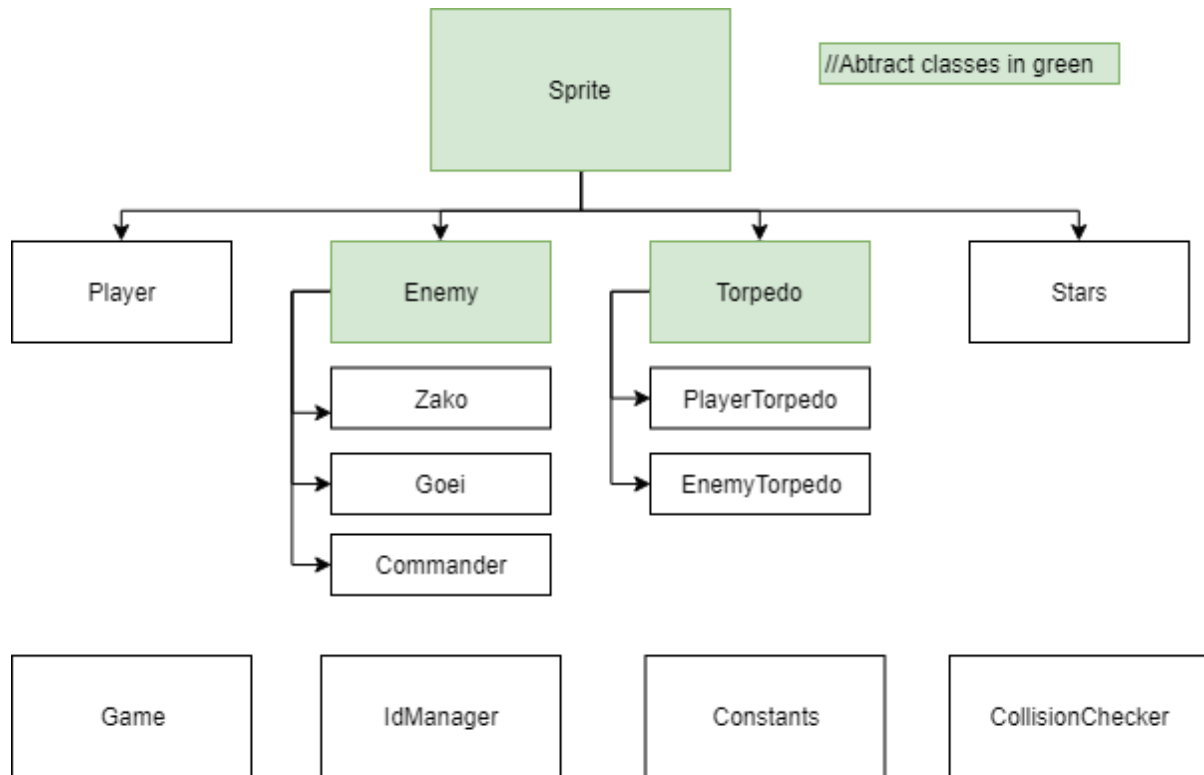


Fig 1. *Classes Design scheme*

Game

This is the most important class as it contains the **main thread** of the videogame. Inside it we find the *main while*: it will keep the game in a loop forever. Inside the main while, there are two whiles; one executed when the game is waiting for the user to click the *new game* button and other while called *game while* that processes a frame every time it is executed.

Game also is in charge of updating part of the stats that appear on the right side of the board, the main controls to move the player, the generation of the enemies, the change of level and calling the enemies to attack, for instance.

Sprite

All the sprite classes had similar characteristics, so the most appropriate solution was to create an abstract mother class called **Sprite**. It is the mother class for *Enemy*, *Player*, *Star* and *Torpedo*.

Enemy

Abstract class mother of **Zako**, **Goei** and **Commander**. It contains all the common methods between of the enemies: animations, swarm updaters and attacks, for example.

Zako, Goei, Commander

One class per type of enemy with their own characteristics: their own sprites and lifes.

Player

Contains the code related to the sprite controlled by the user. Here can be found the player **lifes system**, some configurations for its **torpedoes**, conditions to pass the level, etc.

Torpedo

An abstract mother class of *PlayerTorpedo* and *EnemyTorpedo*. It contains the similar characteristics theses classes have.

PlayerTorpedo & EnemyTorpedo

Each class has overwritten some methods such as the movements and the skins.

CollisionChecker

This class is called in every frame to check the collisions between sprites given two arrays of them (it also admits individual sprites or arrays of arrays of sprites, for example). If a collision is detected, this class will call the *collision* method in the affected sprites.

Constants

An entire class for data and predefined variables. Some important constants like the fps or the board width are declared here. This is useful if we want to change a game constant because we only have to modify it in one place.

We have also written many parts of the program in function of the board dimensions defined in Constants. Therefore, the game can work on different screen sizes and it is *responsive*.

IdManager

To assign each sprite an unique ID and avoid confusions. It uses static methods and variables since we only want one object to prevent id repetition.

Star

Represents the stars sprites. Two layers are created full of objects of this class, each layer with its own speed so it gives a sensation of depth to the player.

2. Most relevant fields and methods

The most relevant methods are related to the movement of the sprites. There are three different methods in the *Sprite* class to move sprites: *moveSpriteTo*, *changeSpritePos* and *changeSpritePosDxDy*. These functions are vital for the game, as they are used constantly and have to be as optimised as possible.

moveSpriteTo instantly move one sprite to another without changing the skin. The *x* and *y* it already have are replaced by the others given to the method.

changeSpritePos changes the sprite position according to the coordinates system given in the project. It has two arguments: one for the direction and another for the modulus. The skin is changed to look in the movement direction.

changeSpritePosDxDy variates the sprite position by the given values of displacement. It is different to *changeSpritePos*, it has two arguments: the variation in *x* and *y* instead of a direction and a modulus. It does not use the coordinates system defines in *Constants* and the skin does not change.

generateRouteToSwarmPos, located in *Enemy*, generates a path for the enemies to return to their position of the swarm. The method works with the movement system *changeSpritePosDxDy*. A variable precision has been created to adjust the smoothness.

update is called in the game while in each frame. Every sprite has this method. Depending on the type of *Sprite*, this method is set differently updating different stats. All the actions that a sprite might have to do in a frame are inside the method *update*.

collision is a method in charge of executing an algorithm when a collision is detected. It is very related to the lives system, as in every collision a life is lost. When a collision is given and lives reach zero, the sprite is destroyed.

Other method related to the movement that is constantly used is ***getPath*** in *Constants* class. Depending on the sprite that uses this method and its *pathNumber*, *getPath* will return one of these paths defined in *Constants*. These are directions that sprites will follow. In case 0, we are using the initial animation, that is different for each enemy type. The other numbers refer to special movements the enemies use to attack the player. Moreover, the creation of new paths is very simple; we only have to increase the variable *availableNumberOfPath* (located at the end of *Constants*) by each new path we declare.

Special mention to the implementation to get the sprites skin with our method *getSkin* located in *Constants* class. Sprites call this method every frame to change of skin (if they need to). The method works depending on the instance of sprite, the *lives* of the sprite and the *direction* the sprite is pointing. We use a *String* system that chooses the right image depending on the direction selected.

3. Most relevant algorithms

The most relevant algorithm that has been used is the main while located in the *Game* class. Once the game is initialized, the program enters a this loop that do an iteration in each frame. We use this loop to update all the sprites, including their position and their sprite image. Every sprite has its own update function to update their characteristics in each frame. To make the loop iterate, we created an FPS Controller inside it that uses Java Time sets.

Other interesting system to mention is the creation of the Torpedos. We created two arrays of torpedoes: one for the player and the other for the enemies. The array of torpedoes of the player is passed to him as an instance. The array of torpedoes of the enemies is passed to each object of *Enemy*. All enemies have the same array as they share an static variable that acts as an index. This way enemies are able to use the torpedoes in order.

In the demo, the player could shoot torpedoes too fast that could win easily. An algorithm has been developed inside the ***shootTorpedo*** method in *Player* class to avoid this happening. There is a limit of torpedoes that can be shot in a period of time.

frameCounter is a field present in every sprite that is increased in each frame. We have used this counter to slow down certain parts of the code. It can be seen in the method ***moveToNextPosition*** in *Sprite*.

Finally but not least, the system inside *CollisionChecker* that controls the collisions between sprites. It uses five methods to check collisions, each one asking

different parameters. This way, all collisions are unified at the same place and less modifications have to be made.

4. Performed work

We managed to achieve all the steps we are asked for with a lot of effort, trying to keep the code as simple as possible. When the program is started, the player can create a new game, inserting his name. Once he has finished, the enemies will appear and do their initial animation. After this, they will start doing the swarm movements and some of them will attack the player. If the player gets hit, he loses one life (represented by player sprites on the bottom left). The player can move with the left and right keys as well as shoot torpedoes with space.

If all enemies are destroyed, the player will pass to the next level. The enemies objects that have been “defeated” are invisible and their collisions are inactive but the sprite objects are not destroyed. Since there are not references to the previous sprites, the Java Garbage collector erases the objects since they are not currently being used.

In addition to the project, we have created stars that move in the background of the game. They have their own class *Star*, whose mother is *Sprite* class. Stars objects are created on the *Game* class with predefined coordinates and speeds. The star sprite image has been created on Photoshop in a .png format to use transparencies.

Another extra feature we have added are Mega Torpedoes. They have been created in the *PlayerTorpedo* class as the other normal torpedoes. The player has a limited amount of these torpedoes in each level and can be used pressing the key *up*. These torpedoes destroy all the enemies since they have many lives.

Very important to mention that the project has been programmed to be responsive. The game can adapt to any screen due to the implementation of the code and *Constants* class.

As the demo showed, commands have been implemented to the game. These commands are deactivated when the game starts to fit the project parameters. Before starting a new game, the player can type “*god*” or “*fastcontrols*” in the bottom right bar. “*god*” will give the player a significant larger amount of lives while “*fastcontrols*” will activate the system we have created to have faster controls over the player.

We did not include the commander attacks with other enemies since we found too complex the process of finding near enemies that were in the swarm and near the commander. However, we build a system so that any sprite can tell other to attack with the same pattern, indeed, sometimes the enemies will attack in a swarm following the same attack pattern.

5. Conclusion

During this project we have learnt a lot of java and object oriented programming, although we have done so by working very hard.

The journey has been full of obstacles that have been solved successfully. As a team we came across many problems: moving the sprites in the swarm at the same time, creating the attack system, a collision detector and solving all the bugs that came out.

Github was very helpful at the beginning but then merge errors came. We had to organise ourselves to avoid them. There were times code got overwritten, leaving errors and more confusion. At the end this became **positive** as now we are more prepared to deal with these problems.

6. Personal comments

First of all: we found of great help the article "*Why getters and setters are the evil*". In the middle of our project development we found that many parts were very bad implemented and others were structured programming looking like object oriented programming. We drastically reduced the public getters and setters that were used outside the class.

We think that Git should be taught in this subject since the university has free premium licenses of GitHub and it is useful both for our project development and for our future job. Everyone uses git.

We found very annoying that the player movements were so slow because the board only detected when a key has been typed and not when it has been *pressed* and *released*.

To sum up: we liked a lot developing this game -despite the difficulties- because we have learned more than what we expected about object oriented programming and java.